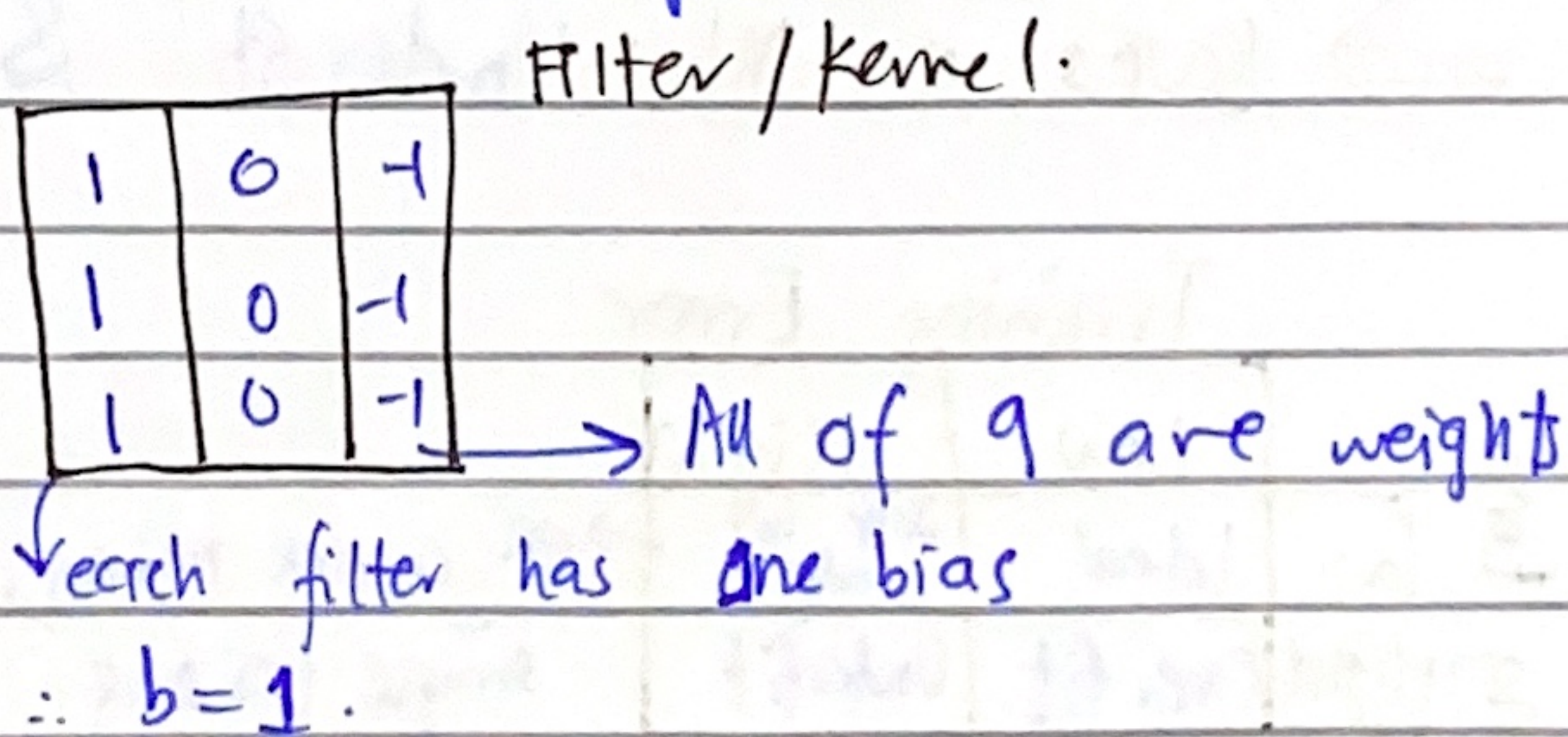
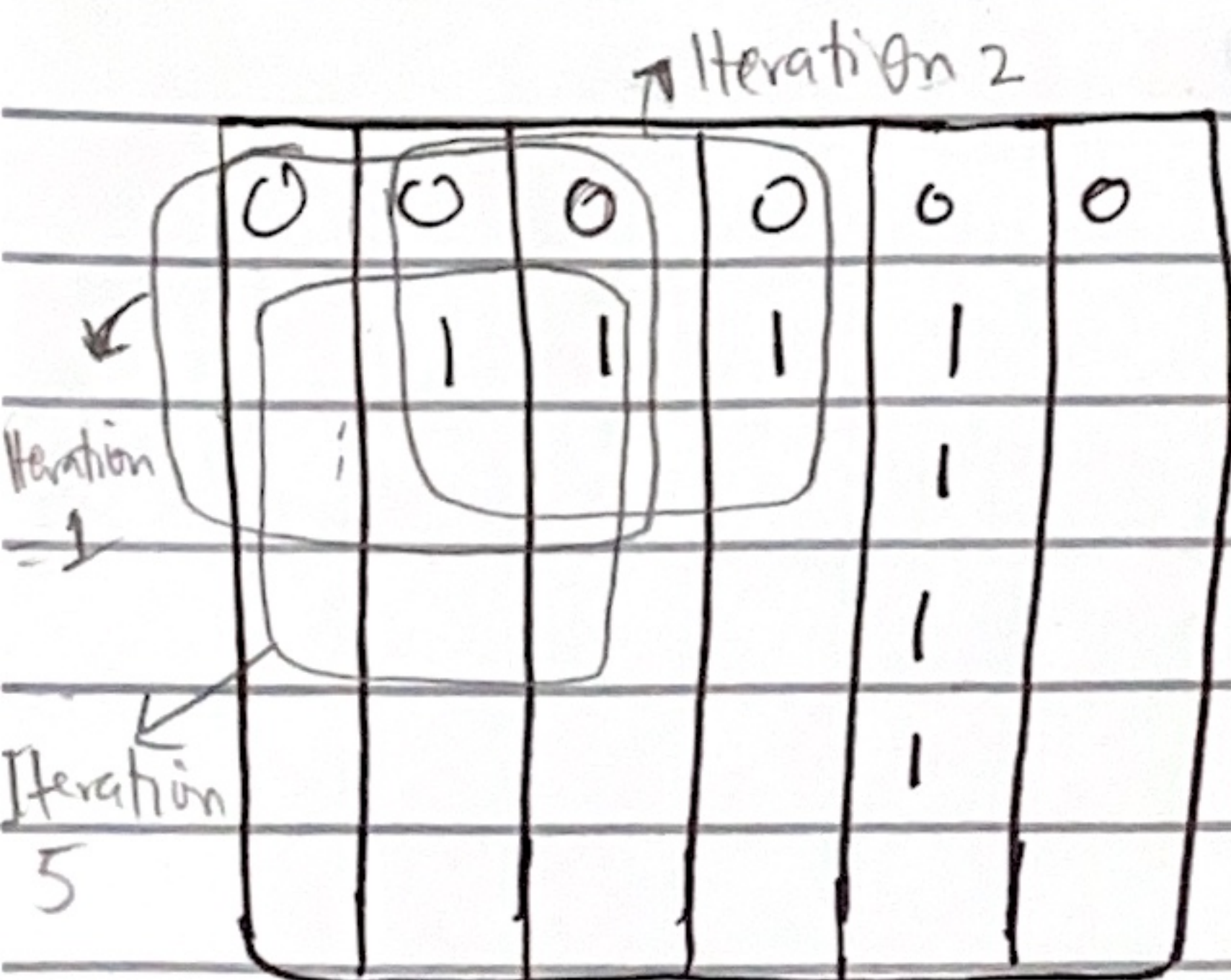


LECTURE-22

Monday
27/4/26

CNN:

- To process image using ANN; we'll be having approx. 100,000 trainable parameters (weights & biases) for just a small 28×28 image & two hidden layers only with 100 neurons each. This will require a lot of time, power & space.
- Moreover when we convert 28×28 image into 1D array (to be feeded into ANN first layer); we lose spacial info which tells correlation neighbours etc b/w pixels.



Here $Stride = 1$ (means we'll jump 1 row or column in each iteration).

After four iterations (in first row for columns); we'll move just 1 step down on row 2 and start from 1st column again for 5th iteration.

Convolution layer:

Working: we'll place the filter on image & do dot product (corresponding multiplication to get 9 values & add of them to get a single value). Put those values into a matrix.

~~Slide 0~~
Iteration 1:

$$\begin{matrix} \text{image Slice 1} & \text{filter} \\ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} & \cdot \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = \begin{matrix} (0 \times 1) + (0 \times 0) + (0 \times -1) + \\ (0 \times 1) + (1 \times 0) + (1 \times -1) + \\ (0 \times 1) + (0 \times 0) + (0 \times -1) \end{matrix} \\ & & = -1 \end{matrix}$$

Now add bias into above dot product's answer.
 $= -1 + 1 = \boxed{0}$ answer of slide 0.

Iteration 2:

~~Slide 1~~ Move the filter 1 column forward.

$$\begin{matrix} \text{image Slice 2} & \text{filter} \\ \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix} & \cdot \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = 0 + 1 - 1 + 0 \end{matrix}$$

★ Add bias = 1

$$= 0 + 1 = \boxed{1} \text{ answer of Slide 1}$$

$$\begin{bmatrix} 0 & 1 & 0 & 2 \\ 0 & 1 & -1 & 2 \\ 1 & 1 & -2 & 1 \\ 1 & 1 & -1 & 1 \end{bmatrix}$$

Output Feature Map

⇒ Output feature map: By adding bias to dot product while sliding the filter over image (on a specific slide rate) we'll get 16 values here which we'll keep ~~adding~~ plugging in a 4x4 (in this case) Matrix known as: output feature map.

- After this we'll pass the output feature map into an activation func. Relu. It gives answer = 0 if input = 0 or -ve and answer = same number if input = +ve

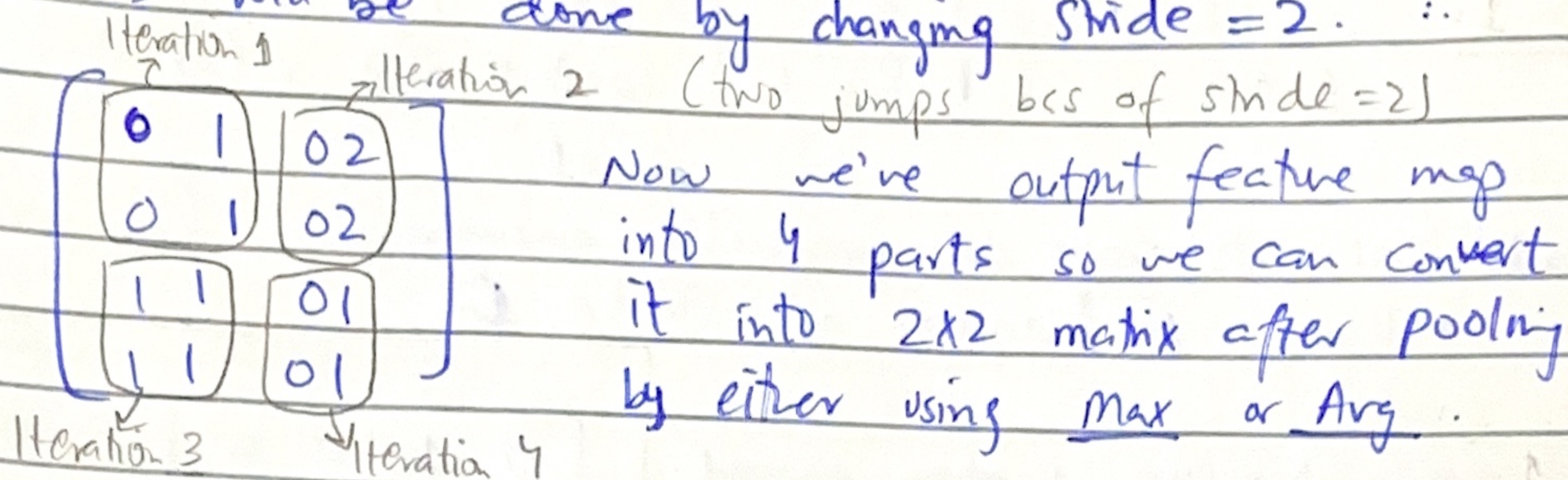
$$\begin{bmatrix} 0 & 1 & 0 & 2 \\ 0 & 1 & -1 & 2 \\ 1 & 1 & -2 & 1 \\ 1 & 1 & -1 & 1 \end{bmatrix} \xrightarrow{\text{after Relu}} \begin{bmatrix} 0 & 1 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix} \begin{matrix} \text{Matrix} \\ \text{after} \\ \text{Relu.} \end{matrix}$$

Pooling:

Working: Now after we get answer from Relu; we need to decrease the height and width (not depth in 3D) by halving it.

- Means output feature map of $4 \times 4 \rightarrow 2 \times 2$.

This will be done by changing $slide = 2$. $\therefore$



Max Pooling: Choose maximum value from Iteration's block

Iteration 1 : 1

Iteration 3 : 1

Iteration 2 : 2

Iteration 4 : 1

* In average pooling; we'll add all values in that block & divide by 4

• Matrix after max Pooling: $\begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix}_{2 \times 2}$ size is halved

- Now we'll flatten this 2×2 into a 1D vector so it can be feeded to ANN.

vector = $[1 \ 2 \ 1 \ 1]$

• These are now basically for inputs feature for ANN (x_1, x_2, x_3, x_4) for 6×6 image whereas if we'd have directly used ANN w/out convolution & Pooling; we'd have 36 inputs (6×6)

• Moreover we've our features extracted first here and then the flatten input is feeded to ANN.

Perceptron

	h_1	h_2	output
x_1	0	0	0 — ans 1
x_2	0	0	0 — ans 2
x_3	0	0	0 — ans 3
x_4	0	0	0 — ans 4
	0	0	0 — ans 5

Softmaxing: (Relative Freq.)

We'll do this after we'll have all answers of output after the activation func.

Suppose: $O_1 = 0.43$, $O_2 = 0.61$, $O_3 = 0.49$

$$\sum \text{Output} = 0.43 + 0.61 + 0.49 = \boxed{1.53}$$

Relative freq: ~~0.1~~ $C_1 = 0.43/1.53$, $C_2 = 0.61/1.53$, $C_3 = 0.49/1.53$
(sum of $C_i = 1$) $C_1 = 0.28$, $C_2 = 0.39$, $C_3 = 0.32$

* As each output 'O' is linked to a class 'C' $\therefore$ we've our probabilities. Key Class 1 key chances are 28%.

Final Step would be since C_2 has highest probability (39%) $\therefore$ model predicts that 6x6 Image belongs to C_2 .

Error: Since model won't predict right class on first attempt $\therefore$ now since ^{final} answer's wrong so we'll be updating the filter (i.e. weights) and then re-run the filter & convolution on original image.

- Suppose we use 8 filters $\therefore$ we'll now have OUTPUT FEATURE MAP of: $4 \times 4 \times 8$ (3D stack) 4 = height
4 = width
8 = depth
- After this when we'll do pooling then we'll get a $2 \times 2 \times 8$ matrix as only height and width are halved (not depth).
- After pooled matrix get flatten; we'll have a 32 ($2 \times 2 \times 8$) vector input to be feed to Perceptron ANN.

Size of Output Feature Map = $\frac{\text{Image dimension} - \text{Filter dimension}}{\text{Stride}} + 1$

eg: 6x6 Image with 3x3 filter

$$\text{Size} = \frac{6 - 3}{1 \rightarrow \text{stride}} + 1 = \boxed{4} \rightarrow \text{means } 4 \times 4 \text{ matrix.}$$